

Architectural Optimization Plan:

`customer_attributes.segment`

Executive Summary

As the `customer_attributes.segment` table scales toward 1 TB, the current schema design introduces severe scaling risks for our CPU-constrained Aurora PostgreSQL environment. The current setup (modulus 10 partitioning, redundant indexing, and unoptimized column ordering) will lead to memory saturation, heavy disk I/O, and excessive CPU overhead during background maintenance.

This document outlines three structural optimizations to reduce CPU load, cut index storage by ~50%, and ensure active working sets remain pinned in memory as we scale.

1. Partitioning Strategy: Scaling to Modulus 32

Current State: Hash partitioned on `persona_id` with 10 partitions. At 1 TB, this yields ~100 GB per partition. **Target State:** Hash partitioned on `persona_id` with **32 partitions**.

Justification:

- **Memory Efficiency:** PostgreSQL requires active indexes to fit within `shared_buffers` (RAM) to avoid CPU-intensive disk I/O. By expanding to 32 partitions, we reduce the partition size to ~31 GB at the 1 TB mark, ensuring the active indexes comfortably fit in memory.
- **Mathematical Hashing:** PostgreSQL's internal hashing algorithm distributes data most evenly when the partition count is a power of 2. Moving from 10 to 32 prevents uneven data skew (hot partitions) and simplifies future splitting operations.
- **Optimal Pruning:** Because application queries strictly filter by the hash key (`persona_id`), the query planner will perfectly prune 31 partitions from the execution plan, requiring near-zero CPU to plan and execute.

2. Index Optimization: Eliminating Redundancy

Current State: Two indexes duplicate the same string data: the implicit composite Primary Key index (`persona_id, segment_name`) and a secondary index (`segment_name`). **Target State:** Drop the secondary `segment_name_index`.

Justification:

- **B-Tree Left-to-Right Rule:** Because our primary access pattern always includes `persona_id` (the leading column of the PK), the composite Primary Key index perfectly covers all lookups. The secondary index provides zero read performance benefit.
- **Halving Write Penalty:** Dropping this redundant index cuts the CPU and I/O penalty for every `INSERT`, `UPDATE`, and `DELETE` in half.
- **Storage Recovery:** This eliminates approximately 11 GB of wasted storage per partition (scaling to hundreds of gigabytes at 1 TB) and drastically reduces the workload on background `autovacuum` sweeps.

3. Schema Reordering: Eliminating Padding Bloat

Current State: Fixed-width 8-byte columns (`timestampz`) are positioned after variable-length `varchar` columns. **Target State:** Reorder columns to place fixed-width columns before variable-width columns.

Justification:

- **Data Alignment (Column Tetris):** PostgreSQL aligns 8-byte data types on 8-byte memory boundaries. When an 8-byte column follows a variable-length string, PostgreSQL silently inserts up to 7 bytes of dead padding per row to force alignment.
- **Storage Savings:** Over billions of rows, a few wasted bytes per row compound into dozens of gigabytes of dead space. Placing all `timestampz` columns together before the `varchar` payload eliminates this hidden padding entirely, saving storage and RAM.

Target DDL Specification

SQL

```
CREATE TABLE customer_attributes.segment (  
  -- Fixed-width columns first (8-byte alignment)  
  segment_last_update_tmstmp timestampz NULL,  
  created_tmstmp timestampz NULL,  
  last_updated_tmstmp timestampz NULL,  
  
  -- Variable-width columns  
  persona_id varchar(34) NOT NULL,  
  segment_name varchar(100) NOT NULL,  
  segment_id varchar(50) NULL,  
  persona_id_type varchar(50) NULL,  
  flow_id varchar(10) NULL,  
  variant_id varchar(10) NULL,
```

```
    CONSTRAINT segment_pk PRIMARY KEY (persona_id, segment_name)
) PARTITION BY HASH (persona_id);
```

-- Note: No secondary indexes are created.

-- Partitions (Example 1 of 32)

```
CREATE TABLE customer_attributes.segment_p0 PARTITION OF
customer_attributes.segment FOR VALUES WITH (modulus 32, remainder 0);
-- Continue through remainder 31...
```